



Styled Components

O [Styled-Components](#) é uma biblioteca para React e React Native que permite usar estilos ao nível de componente na sua aplicação. São escritos utilizando JavaScript e CSS, assim o modo de estilizar um componente utilizando esta biblioteca também é conhecido como CSS-in-JS.

Diferenças entre Styled-Components e CSS

Algumas diferenças entre essas duas formas de estilização são:

- com o styled-components não é necessário manipular nomes de classes, já que a biblioteca gera nomes de classes de forma automática ao renderizar o componente no navegador, eliminando a ocorrência de *bugs* por erros de ortografia e digitação;
- a estilização dinâmica se torna mais simples, sendo que é possível adaptar estilos baseado em props;
- a manutenção do código se torna mais fácil já que cada componente terá seu próprio estilo, sem afetar os demais;
- componentes são renderizados juntamente com a página, dessa maneira, apenas a estilização que será utilizada será injetada. Será carregado somente o necessário.

Instalando a biblioteca em seu projeto

Para começar a usar o styled-components em um projeto é muito simples. Basta realizar a sua instalação através dos seguintes comandos:

```
yarn add styled-components  
ou  
npm install styled-components
```

Como estamos utilizando TypeScript o projeto, devemos instalar também o [package de definição de tipos](#) do styled-components:

```
yarn add @types/styled-components -D  
ou  
npm install @types/styled-components -D
```



Criando componentes

Diferentemente do uso do CSS, onde as estilizações eram aplicadas através de classes e ids, por exemplo, com o styled-components são criados componentes que recebem a estilização.

Inicialmente, é criada uma constante, no arquivo de estilos, que possui o comando *export*, possibilitando que seja utilizada em outro arquivo.

Após declarar a constante, deve ser digitado *styled* e então escolher uma opção dentre todos os elementos do HTML que a biblioteca nos permite utilizar.

Toda a estilização é feita utilizando [Template Literals](#), que permite usar múltiplas linhas, expressões embutidas, entre outras opções que você pode consultar na documentação.

```
import styled from "styled-components";

export const Button = styled.button`
  width: 200px;
  height: 50px;
  color: #FFF;
  background-color: #7159c1;

  &:hover {
    background-color: #0000FF;
  }
`;
```

Para utilizar esse componente de estilização criado, basta importar dentro do arquivo de renderização.

```
import React from "react";
import { Button } from "./styles";

function App(){
  return <Button />
};

export default App;
```



Uso de JSX

A utilização do styled-components torna a hierarquia visual do seu código mais semântica.

Sem a utilização dele teríamos o componente mostrado abaixo:

```
const Header = ({userName, userEmail, logOut}) => {
  return (
    <div className="header">
      <div className="header-brand">
        <div className="logo">
          <img width={45} height={45} src={logo} className="logo" alt="Brand"/>
        </div>
        <h1>Brand name</h1>
      </div>
      <div className="header-user">
        <span className="header-user-name">{userName || userEmail} </span>
        <span className="header-user-logout" onClick={logOut}>(log out) </span>
      </div>
    </div>
  );
};
```

Já utilizando essa biblioteca, esse mesmo componente poderia ficar desta forma:

```
const Header = ({userName, userEmail, logOut}) => {
  return (
    <Header>
      <Brand>
        <Logo>
          <img width={45} height={45} src={logo} className="logo" alt="Brand"/>
        </Logo>
        <h1>Brand name</h1>
      </Brand>
      <Welcome>
        <UserName>{userName || userEmail} </UserName>
        <LogOut onClick={logOut}>(log out) </LogOut>
      </Welcome>
    </Header>
  );
};
```



Alternando estilos

Uma vez que o styled-components nos permite passar uma função como valor do estilo, é possível alterar o estilo de um componente atrás das props que são passadas a ele, funcionando da mesma forma como é feito no HTML onde adicionamos classes para estilização.

Como resultado se obtém uma redução em linhas de código.

```
.button{
  color: #FFF;
  background: #039BE5;
  box-shadow: 2px 5px;
  cursor: pointer;
}
.button.default {
  box-shadow: none;
  background: none;
  color: #999;
}
.button.default:hover {
  background: #EEE;
}
.button:hover {
  background: #0388CA;
}

const Button = styled.button `
  color: ${props => props.default ? "#999" : "#FFF"};
  background: ${props => props.default ? "none" : "#039BE5"};
  box-shadow: ${props => props.default ? "none" : "2px 5px"};
  cursor: pointer;

  &:hover {
    background: ${props => props.default ? "#EEE" : "#0388CA"};
  }
`;
```

Outra vantagem dessa biblioteca referente a estilização é que não são apenas elementos DOM que podem ser estilizados. Ela permite que seja passado qualquer componente.

```
const Message = styled.div`
  display: flex;
  width: 35%;
  border-radius: 10px;
  border: 2px solid;
`;

const Success = styled(Message)`
  background: #78cc78;
  border-color: #79cc53;
  text-shadow: 0 0 3px #000;
`;
```



Adicionando atributos em componentes

O styled-components possui o método `attrs`, que é utilizado para adicionar atributos aos componentes de estilização criados.

```
<input
  id="firstName"
  name="firstName"
  placeholder="Your first name..."
  type="text"
/>
```

```
export const Input= styled.input.attrs({
  id:"firstName",
  name:"firstName",
  placeholder:"Your first name...",
  type:"text"
})`
margin-top: 2px;
background-color: #47e500;
```

Quando seu componente for importado, não precisará passar os atributos novamente.

```
import React from "react";
import { Input } from "./styles";
```

```
function App(){
  return <Input />
};
```

```
export default App;
```

Contudo, não é possível disparar eventos (onClick, onChange,...) com ele, sendo necessário utilizar o disparo de eventos diretamente no componente.

```
import React from "react";
import { Input } from "./styles";
```

```
function App(){
  return <Input onChange={handleChange}/>
};
```

```
export default App;
```